A PRACTICAL GUIDE TO STRATA

# Mac Guide

## Installation and First Experiments

*Step by step from scratch, no programming needed*



cd ~/sirisys · 24 · 57 · 6 · 12 · 16 · same ledger: YES

**Juan A. Pérez-Gómez**

Version 13.5 · 13 November 2026

Independent interpretation · no affiliation with SIRISYS

# Contents

Step 1 · Download the project . . . . . 1
Step 2 · Open the Terminal . . . . . 2
Step 3 · Check whether you have Python . . . . . 2
Step 4 · Go to the folder . . . . . 2
Step 5 · Install the pieces . . . . . 3
Step 6 · Check that everything works (free, no model) . . . . . 3
Step 7 · Choose the model . . . . . 3
Step 8 · The short report test . . . . . 4
Step 9 · Suggested experiments . . . . . 4
Step 10 · Your results . . . . . 5
If something fails . . . . . 6
Summary . . . . . 7

A step-by-step guide from scratch for STRATA, an independent software interpretation of SIRISYS (version 13.5). No programming needed, and no prior use of the Terminal.

### Before you start

You will need:

- Your Mac and an internet connection.
- The STRATA repository on GitHub: github.com/juanan-prz/STRATA (step 1 explains how to download it).
- For the experiments with a model: Ollama, free and local (recommended), or an Anthropic API key (optional). The functional tests need neither.

The first time takes about 30 minutes of installation. After that, each session starts in a minute.

GOLDEN RULE

Copy and paste the commands exactly as they are, without changing anything. One extra space or a misplaced character and they fail. Copying and pasting is the right way, not cheating.

A guide to an independent interpretation in software. STRATA is not an official SIRISYS work and implies no affiliation with SIRISYS.

# Step 1 Download the project

- Open https://github.com/juanan-prz/STRATA in Safari.
- Click the green **Code** button and then **Download ZIP**.
- Open the downloaded ZIP (double-click): a folder appears.
- Move it to your **home folder** (the one with your user name and a house icon) and rename it to exactly `sirisys` (lower case, no accents or spaces).

CAUTION

**Do not put it in Documents or on the Desktop.** If iCloud syncs them with «Optimise Mac Storage», it may keep the files only in the cloud; with Wi-Fi off, the programs cannot read them and the test comes out empty without warning.

Inside, together and with no subfolders, you should find these files (among others):

| File | What it is for |
|---|---|
| `Sirisys_Framework_v13_5.py` | The system. |
| `Sirisys_Server.py` · `sirisys_loader.py` | Start experiment mode, with the live view. |
| `Sirisys_Live_Visualizer.html` | The live view of the field. |
| `Sirisys_Static_Visualizer.html` | The inspector: examines a saved instance afterwards. |
| `run_report.py` | Measured runs: writes a report with their results. |
| `field_topology.py` · `sirisys_measures.py` | Measurements the reports use. They also check that the model answers. |
| `test_*.py` · `sirisys_testkit.py` | Functional tests: five files and what they share. |
| `measure_phase3.py` · `live_c2.py` · `soak.py` | Measurements: criteria, real percepts and a long life. |
| `requirements.txt` | The list of pieces to install. |

Always download on the computer, not on a phone. If the files pass through a phone's clipboard or a rich-text editor, the quotation marks get corrupted and the code stops working.

When a new version comes out, rename the old folder (for example, `sirisys_v134`) before downloading the new one: that way you keep its results and the files do not mix.

# Step 2 Open the Terminal

The Terminal is a window where you type commands instead of clicking. Press Command + Space, type Terminal and press Enter. A window opens with a blinking cursor. You cannot break anything by typing there: if a command does not exist, it simply says so.

# Step 3 Check whether you have Python

Copy this into the Terminal and press Enter:

```
python3 --version
```

If it answers Python 3.10 or higher (for example 3.12.4), you have it: skip to step 4. If it says `command not found`, offers to install developer tools, or the version is older: go to python.org/downloads, click the yellow Download Python button, install it like any other application, close the Terminal, open it again and repeat the command.

# Step 4 Go to the folder

```
cd ~/sirisys
ls
```

The first goes into the folder; the second lists what is inside. It should list the files from step 1. If you see them, you are on track.

## Step 5 Install the pieces

```
pip3 install -r requirements.txt
```

You will see a lot of text scroll by: that is normal and takes a couple of minutes. When you can type again, it has finished. If a red warning says `externally-managed-environment` (it usually happens with Homebrew's Python), use this line instead:

```
pip3 install --break-system-packages -r requirements.txt
```

## Step 6 Check that everything works (free, no model)

Run these five tests, one at a time. They cost nothing and need no internet:

| Command | What it should say at the end |
|---|---|
| `python3 test_sirisys.py` | `SUMMARY: 24/24 passed` (≈ 15 s) |
| `python3 test_phase1.py` | `SUMMARY: 57/57 passed` (≈ 20 s) |
| `python3 test_phase2_guarantee.py` | `SUMMARY: 6/6 passed` (≈ 5 s) |
| `python3 test_phase4.py` | `SUMMARY: 12/12 passed` (≈ 5 s) |
| `python3 test_ontology.py` | `SUMMARY: 16/16 passed` (≈ 5 s) |

If any line says `FAIL`, note which test failed and the lines before it; download the ZIP again on the computer and repeat, since a damaged file is the most common cause. To see the field live:

```
python3 Sirisys_Server.py --play
```

Open http://127.0.0.1:8000 in Safari. To stop it, go back to the Terminal and press Control + C: the instance is saved in the `sirisys_runs` folder and nothing is deleted. To examine it afterwards, open `Sirisys_Static_Visualizer.html` in Safari and choose the `state.json` in its folder.

## Step 7 Choose the model

The model is optional and only translates: it gives readable names to what the field produces and words a question at each approach to the Sombunal. What it says never changes what the field does: a life with a model leaves the same record as one without.

**Option A — Ollama (recommended).** The model runs inside your Mac: free, and nothing is sent out.

- Download it from ollama.com and install it like any other application. It keeps running in the background, with an icon in the menu bar.
- In the Terminal, download a model. For example:

```
ollama pull llama3.1
ollama list
```

`ollama list` should show the downloaded model; if it answers with an error, Ollama is not running: open it. Always use the same model in the runs you intend to compare.

**Option B — Anthropic (optional, for experiments only).** At console.anthropic.com create an account, go to API Keys → Create Key, copy the key (it starts with `sk-ant-`; it is shown only once) and add credit under Billing without turning on automatic reload. A Claude subscription does not include API credit: they are separate. Then, in the Terminal:

```
export ANTHROPIC_API_KEY="sk-ant-your-key-here"
```

It answers nothing: that means it worked. It lasts while the window stays open. Never store the key in a file in the folder.

# Step 8 The short report test

```
python3 run_report.py --cycles 20 --seeds 1 --backend ollama --profile minimal
```

(With Anthropic, change `ollama` to `anthropic`.) Before it starts, the program makes three real calls to the model. Look at:

| What to look for | What it should say |
|---|---|
| The 16 check lines at the start | All with `[ok ]`. |
| `model probe` | ok, and the seconds each call takes with a run's context. If the model does not answer or answers empty, the program stops here with `STOP` and says why (see «If something fails»). |
| `sample names` | Two bare names, not sentences. «SYSTEM_AM» is a name; «The emergent construct is SYSTEM_AM.» is a sentence. |
| At the end: `model calls` | `failures 0` and `empty 0`. |
| At the end: `same ledger in every arm` | `YES`: the field leaves the same record with the model and without it. |

# Step 9 Suggested experiments

A short sequence, with the `minimal` profile (no narration, no time of day), in this order. The long ones start with `caffeinate -i`: it keeps the Mac awake while the test runs and stops on its own

when it ends. Do not close the Terminal window; on a laptop, keep it plugged in with the lid open.

**1 · The control, no model (a few seconds).** The field with three synthetic percept streams: it checks that the field meets its criteria on your machine. At the end, the `criteria` line should say `PASS` on all three; if one says `FAIL`, run it again with the Mac idle, since one of the measures is how long each cycle takes.

```
python3 run_report.py --backend none --profile minimal --cycles 1000 --seeds 3
mv sirisys_report.json control_report.json
```

The second command renames the report so that the next one does not overwrite it.

**2 · Real percepts, no model (≈ 10 minutes).** The field with your Mac's percepts over a long life: it checks that the field holds on what it really perceives. The result stays in the file (step 10). The field perceives the processor and the memory: close every application first (Ollama too) and do not use the Mac while it runs.

```
caffeinate -i python3 live_c2.py 10000 none minimal > live_none.json
```

**3 · Real percepts with the model, and its replay without it (half an hour to two hours).** It checks that what the model says does not change what the field does. With Ollama running and nothing else open. At the start it says in the Terminal the most time it will take.

```
caffeinate -i python3 live_c2.py 100 ollama minimal > live_ollama.json
python3 live_c2.py 100 none minimal --replay live_ollama.json > live_replay.json
```

The second lives the same life again, with the same percepts, without a model, and takes a few seconds. At the end it should say `same ledger as the replayed run: YES`.

While `live_c2.py` runs nothing else appears on screen: that is normal, it writes the result at the end. If the model stops answering halfway (Ollama closes or hangs), the program stops on its own and says so in the Terminal with `STOPPED`. Start Ollama and run that same command again; the interrupted file is not needed.

### Optional: watch the field live with the model.

```
export SIRISYS_LLM_BACKEND=ollama
python3 Sirisys_Server.py --play --profile minimal
```

Open http://127.0.0.1:8000 as in step 6. Hovering over a construct shows the name the model gave it and, below, its key, the field's own. The model is shown at the top; if it stops answering, the field pauses, the header marks it in red (`watch tripped`) and the event log says why. Never at the same time as a test: stop it with Control + C before starting the next one.

# Step 10 Your results

The `sirisys` folder will now hold four files:

- `control_report.json`, the control;
- `live_none.json`, the real percepts without a model;

- `live_ollama.json`, the real percepts with the model;
- `live_replay.json`, its replay without a model.

They are text files (JSON): each records the run's configuration, the self-adjustment criteria and their verdict, the model's health, the timings and the ledger head. Those with a model also keep every name it gave, with its whole answer. The technical guide (`SIRISYS_Guide_v13_EN.md`) explains what each one contains. To compare results across machines or models, also note:

- the name of the Ollama model you used (`ollama list` shows it);
- your Mac's chip and memory (Apple menu → About This Mac).

# If something fails

| What you see | What to do |
|---|---|
| `command not found: python3` | Python is not installed: go back to step 3. |
| `No such file or directory` | You are not in the folder or files are missing: repeat step 4 and check with `ls`. |
| `externally-managed-environment` | Use the second line of step 5. |
| A test says `FAIL` | Download the ZIP again on the computer and repeat. If it still fails, note the test and its message. |
| `model active: False · STOP: … not available` | Ollama is not running (`ollama list` gives an error: open it and repeat), the model is not downloaded or, with Anthropic, the key was not set in this window. |
| `STOP: … answered empty` | The model «thinks» before answering and runs out of room for the name. Use a general-purpose model, for example `llama3.1` (step 7). |
| `STOPPED — model health` | The model stopped answering halfway. Start Ollama and run that command again. |
| `same ledger …: NO` | It is a bug in the code, not a result: keep the output and report it on GitHub. |
| The .json file is 0 bytes | The folder is in Documents or on the Desktop, synced with iCloud: move it to your home folder (step 1) and repeat. |
| `authentication_error / 401 · credit balance is too low` | With Anthropic: the key was copied wrong (copy all of it, without spaces) or there is no credit at console.anthropic.com → Billing. |
| `already open by another writer` | Another SIRISYS is writing in that folder: close the other Terminal window or press Control + C in it. |
| `Address already in use` | Experiment mode is already running in another Terminal window: stop it there with Control + C. |
| The test stopped when the Mac slept | Run it again with `caffeinate -i` in front; on a laptop, plugged in with the lid open. |
| The window seems stuck | Normal: it is working. If it really hung, Control + C. |

# Summary

Each session starts in the folder, and it is worth checking that everything works:

```
cd ~/sirisys
python3 test_sirisys.py
python3 test_phase1.py
python3 test_phase2_guarantee.py
python3 test_phase4.py
python3 test_ontology.py
```

The sequence of experiments from step 9:

```
python3 run_report.py --backend none --profile minimal --cycles 1000 --seeds 3
mv sirisys_report.json control_report.json
caffeinate -i python3 live_c2.py 10000 none minimal > live_none.json
caffeinate -i python3 live_c2.py 100 ollama minimal > live_ollama.json
python3 live_c2.py 100 none minimal --replay live_ollama.json > live_replay.json
```

End of document · Version 13.5 · 13 November 2026